

Assignment 5

Lighting, Materials, and Shading

Computer Graphics Course

C++ | CLion | MiniFB | MicroUI | GLM

Student 1	Natalie Bittar ID: 212586044
Student 2	Adan Awad ID:324302678
Course	Computer Graphics
Assignment	Lighting, Materials, and Shading

This report documents the implementation of lighting and shading in a software renderer. The assignment extends the rasterizer with the Phong reflection model, material and light controls, three shading modes (Flat, Gouraud, and Phong), reflection-vector debugging, and texture mapping.

Report Structure

- Overview
- Part 1: Light Sources and Material Properties
- Part 2: Flat Shading (Diffuse Lighting)
- Part 3: Specular Highlights and Reflection Vectors
- Part 4: Phong Shading (Per-Pixel Shading)
- Part 5: Pair Programming Extensions
- Interaction and Testing
- Conclusion

Overview

In this assignment the previous triangle rasterizer was upgraded into a lighting-aware renderer. Instead of filling each triangle with a constant debugging color only, the renderer now computes surface illumination from a configurable point light and material parameters. The implemented illumination model is the classic Phong reflection model, which combines Ambient, Diffuse, and Specular terms.

The application supports three OBJ models (cube, pyramid, and octahedron), an interactive GUI for changing light position and material properties, and multiple visualization modes. These include Flat Shading, Gouraud Shading, Phong Shading, debug reflection vectors, and texture mapping.

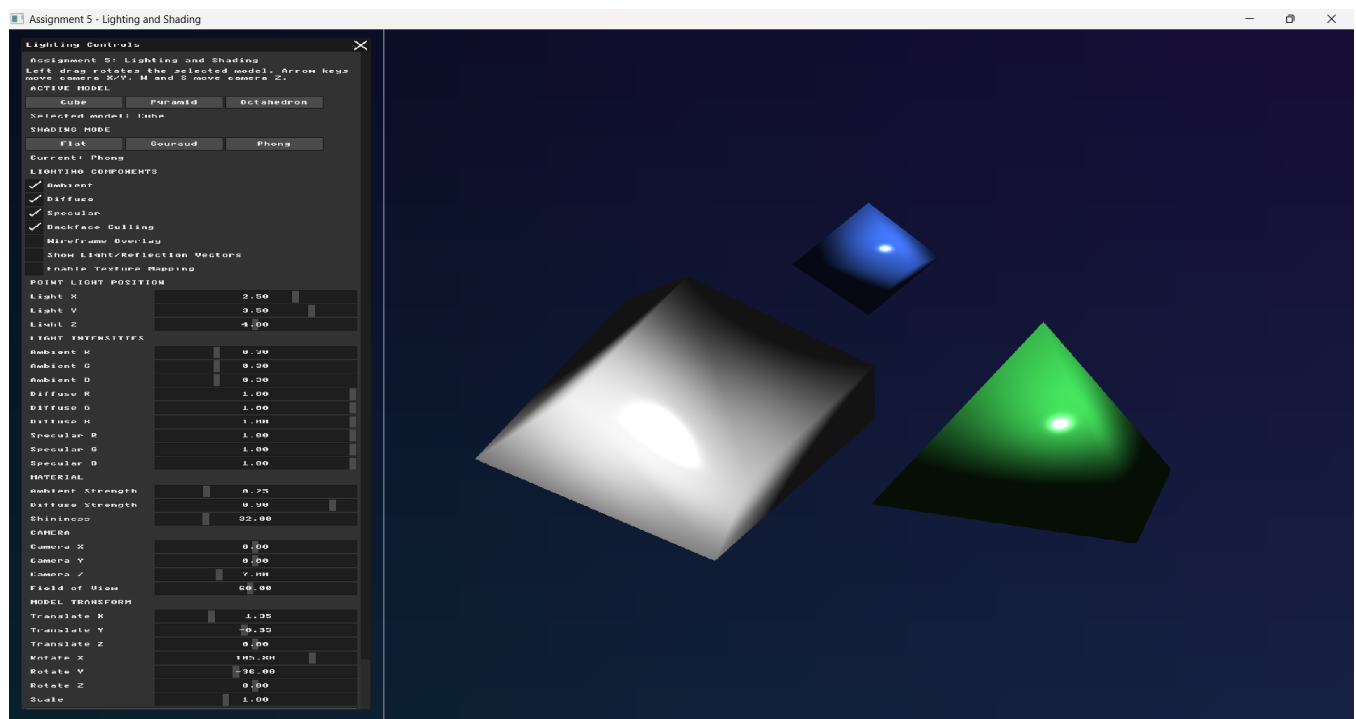


Figure 1. Final rendered scene with Phong lighting enabled.

Part 1: Light Sources and Material Properties

A `PointLight` structure was defined to store the light position and its three color components (ambient, diffuse, and specular). A `Material` structure stores the matching material coefficients as well as the shininess exponent used by the specular term. The GUI exposes sliders for the light position (X, Y, Z), each RGB lighting component, and the material strengths.

```
struct PointLight {
    glm::vec3 position;
    glm::vec3 ambient;
    glm::vec3 diffuse;
    glm::vec3 specular;
};
```

```
struct Material {
    glm::vec3 ambient;
    glm::vec3 diffuse;
    glm::vec3 specular;
    float shininess;
};
```

The first stage was Ambient lighting only. Ambient light is independent of direction and acts as a constant base illumination. It was implemented as the component-wise multiplication of the light ambient term and the material ambient term.

```
ambient = light.ambient * material.ambient;
```

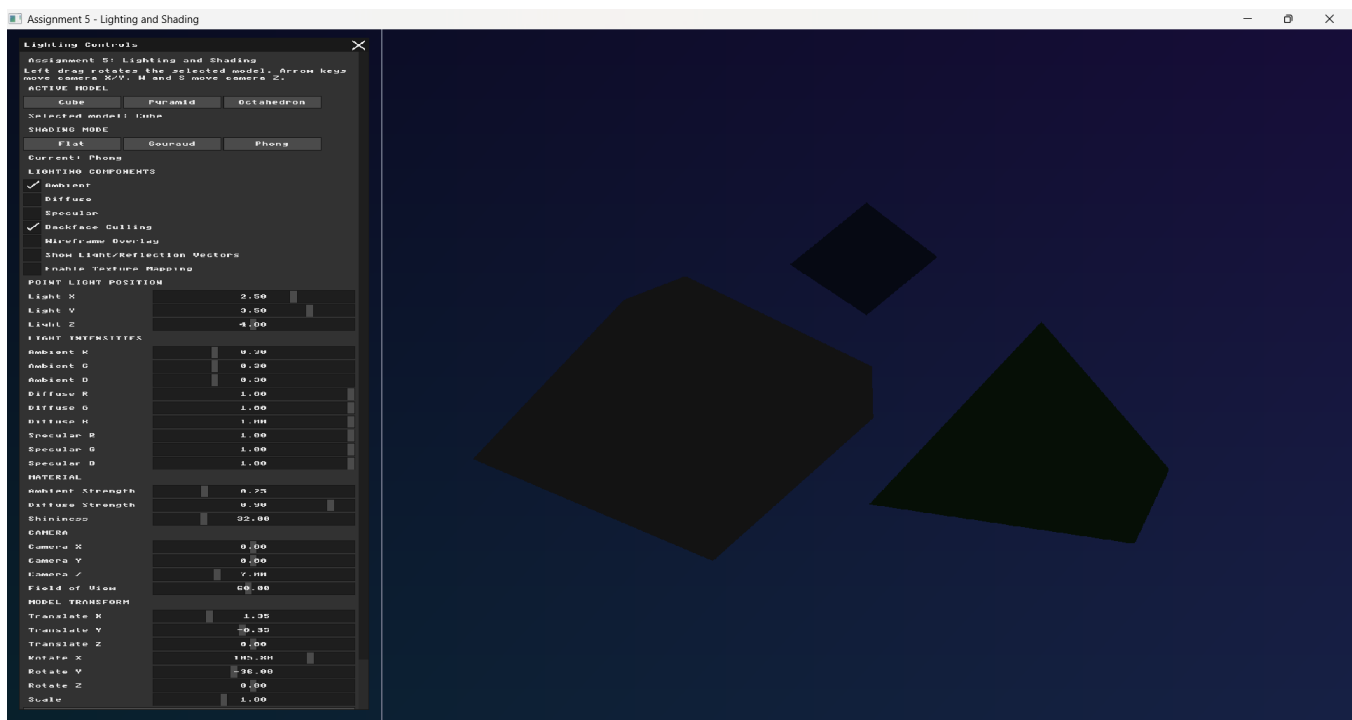


Figure 2. Ambient component only. The objects are visible, but appear flat and lack directional shape cues.

Part 2: Flat Shading (Diffuse Lighting)

Diffuse lighting follows Lambert's cosine law. For each triangle, a face normal was computed and normalized. The light direction was calculated from the triangle center to the point-light position. Flat shading evaluates the lighting equation once per triangle, so every pixel in the triangle uses the same resulting color.

```
L = normalize(light.position - triangleCenter);
N = normalize(faceNormal);
diff = max(dot(N, L), 0.0f);
diffuse = light.diffuse * material.diffuse * diff;
finalColor = ambient + diffuse;
```

This stage makes the shape of each object much clearer than ambient-only rendering. However, the surfaces still look faceted because lighting changes only at triangle boundaries.

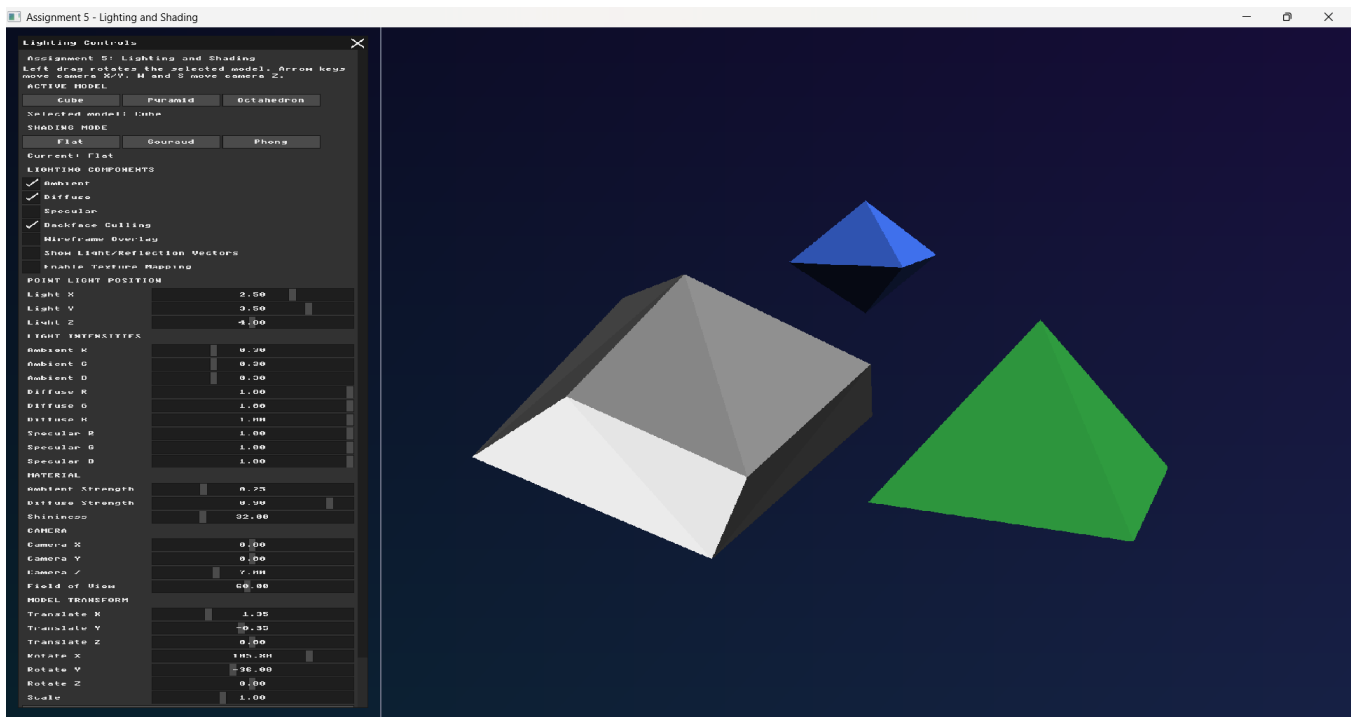


Figure 3. Flat shading. Lighting is evaluated once per triangle, producing a faceted appearance.

Part 3: Specular Highlights and Reflection Vectors

To simulate shininess, the Specular term was added. The reflected light direction is compared with the view direction, and the result is raised to a shininess exponent. When the reflection points toward the camera, a bright highlight appears.

```
R = reflect(-L, N);  
V = normalize(cameraPosition - pointPosition);  
spec = pow(max(dot(R, V), 0.0f), material.shininess);  
specular = light.specular * material.specular * spec;  
finalColor = ambient + diffuse + specular;
```

For debugging, the program can draw the incoming light vector and the outgoing reflection vector from sample points on the model. This helped verify that the reflection direction and highlight positions were correct.

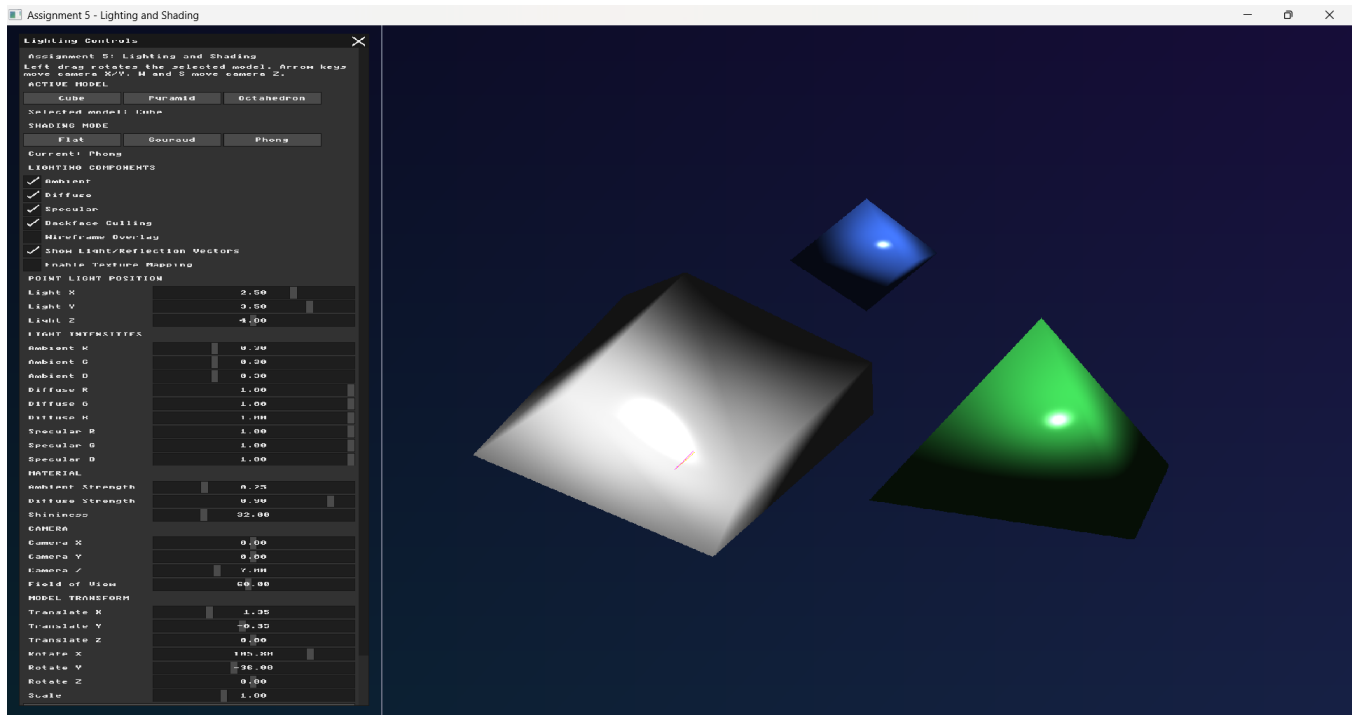


Figure 4. Debug vectors. The incoming light vector and reflection vector are drawn to verify the specular computation.

Part 4: Phong Shading (Per-Pixel Shading)

Phong shading computes illumination per pixel instead of per triangle. During rasterization, the barycentric weights of the current pixel are used to interpolate both the 3D position and the vertex normals. The interpolated normal is normalized and used in the full lighting equation.

```
P = alpha * p0 + beta * p1 + gamma * p2;
N = normalize(alpha * n0 + beta * n1 + gamma * n2);
L = normalize(light.position - P);
V = normalize(cameraPosition - P);
R = reflect(-L, N);
color = ambient + diffuse + specular;
```

Compared with Flat shading, Phong shading creates a much smoother visual result. Highlights are more precise and move continuously across the surface because the full reflection equation is evaluated for every covered pixel.

Figure 1 demonstrates the final Phong-shaded output. The cube and pyramid show smooth light falloff and clearly visible specular highlights.

Part 5: Pair Programming Extensions

Gouraud Shading

Gouraud shading is an intermediate method between Flat and Phong shading. The lighting equation is computed at the three vertices of each triangle, and the resulting colors are interpolated across the triangle interior using barycentric coordinates.

```
c0 = shade(vertex0);
c1 = shade(vertex1);
```

```

c2 = shade(vertex2);
pixelColor = alpha * c0 + beta * c1 + gamma * c2;

```

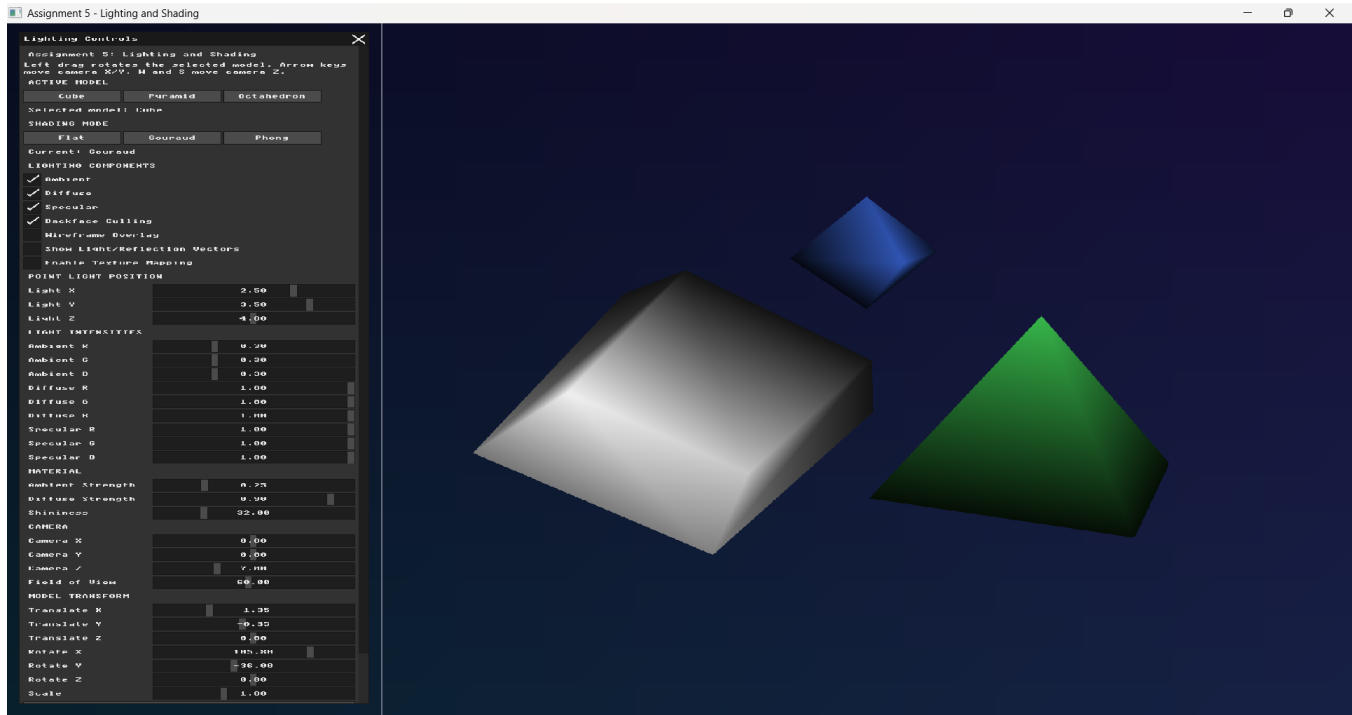


Figure 5. Gouraud shading. Vertex colors are interpolated across each triangle.

Visual comparison: Gouraud shading is smoother than Flat shading, but its specular highlights can be less accurate. If a highlight lies between vertices, it may appear weaker or disappear. Phong shading preserves sharper and more stable highlights because the specular term is evaluated per pixel.

Texture Mapping

The OBJ loader was extended to read texture coordinates (vt). During rasterization, the UV coordinates are interpolated at each pixel using barycentric coordinates. The sampled texture color is then applied to the diffuse component, while ambient and specular lighting continue to affect the final result.

```

uv = alpha * uv0 + beta * uv1 + gamma * uv2;
texColor = sampleTexture(uv);
diffuse = texColor * light.diffuse * max(dot(N, L), 0.0f);

```

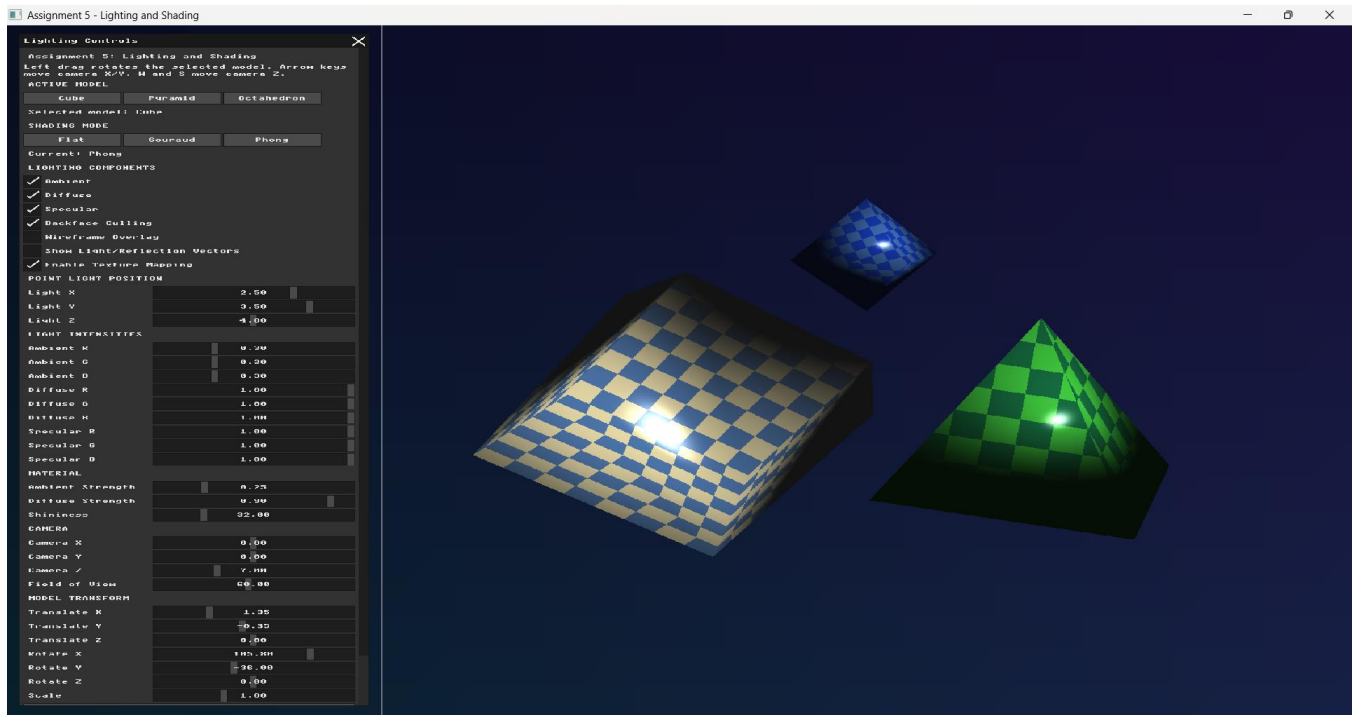


Figure 6. Texture mapping enabled. A checkerboard texture is sampled using interpolated UV coordinates.

Interaction and Testing

The application was configured and tested in CLion using CMake, MiniFB, MicroUI, and GLM. The following behaviors were verified:

- All three models (cube, pyramid, octahedron) load correctly and display valid geometry.
- Ambient, Diffuse, and Specular components can be toggled independently from the GUI.
- The point-light position can be changed in real time using X, Y, and Z sliders.
- Flat, Gouraud, and Phong shading modes can be selected interactively.
- Specular highlights respond to light movement and to the shininess parameter.
- Reflection-vector debugging draws light and reflection directions on the scene.
- Texture mapping correctly applies a checkerboard pattern on the loaded models.
- The final image is rendered through the software rasterizer and updated correctly in the MiniFB window.

Conclusion

This assignment completed the final stage of the software rendering pipeline by adding physically inspired lighting and shading. The renderer now supports configurable point lights, material coefficients, ambient/diffuse/specular illumination, and multiple shading strategies.

Flat shading demonstrates face-based illumination, Gouraud shading interpolates vertex colors efficiently, and Phong shading produces the highest visual quality by evaluating lighting per pixel. The reflection-vector debug mode made verification easier, and texture mapping extended the renderer from shaded solid colors to textured surfaces.